

Logging ✓

→ Datadog

@traces.wrap

def fun1(-a, -b):

return a + b

10 Million times

→ 9:09 AM

→ 9:11 AM

a = 50

b = 70

n = 120

Flag

2 min

Rule / Syntax	Name	Description	Example
Characters & Anchors			
<code>abc</code>	Literal	Matches the exact characters "abc".	<code>abc</code> matches "abc"
<code>.</code>	Wildcard	Matches any single character (except newline).	<code>h.t</code> matches "hat", "hot", "h8t"
<code>^</code>	Start Anchor	Asserts position at the start of the string.	<code>^Cat</code> matches "Cat" in "Cat and dog"
<code>\$</code>	End Anchor	Asserts position at the end of the string.	<code>dog\$</code> matches "dog" in "cat and dog"

h.t

hat, hot

h7t, h@t

✓✓
"Kamla is not a student"
X

↓
"K#a\$"

Quantifiers

<code>*</code>	✓	Zero or More	Matches the preceding item 0 or more times.	<code>ab*c</code> matches "ac", "abc", "abbc"
<code>+</code>	✓	One or More	Matches the preceding item 1 or more times.	<code>ab+c</code> matches "abc", "abbc"
<code>?</code>	✓	Zero or One	Matches the preceding item 0 or 1 time (optional).	<code>colou?r</code> matches "color" and "colour"
<code>{n}</code>	✓	Exact Count	Matches the preceding item exactly <code>n</code> times.	<code>\d{3}</code> matches "123"
<code>{n,}</code>	✓	At Least <code>n</code>	Matches the preceding item <code>n</code> or more times.	<code>\d{2,}</code> matches "12" and "123"
<code>{n,m}</code>	✓	Range	Matches the preceding item between <code>n</code> and <code>m</code> times.	<code>\d{2,4}</code> matches "12", "123", "1234"

Can appear any no of times.

[a-z]*

Any small alphabet

"12345"

X

(KTM)?

$K[a-z]^*T$

KT

$K[a-z]^+T$

KT

(KTM)

KTMM

Character Classes

`[abc]`

Character Set

Matches any single character in the set.

`b[aiu]g` matches "bag", "big", "bug"

`[^abc]`

Negated Set

Matches any single character **not** in the set.

`[^aeiou]` matches any consonant

`[a-z]`

Range

Matches any single character in the range.

`[0-9]` matches any digit

`\d`

Digit

Matches any digit.
Equivalent to `[0-9]`.

`\d+` matches "2025"

`\w`

Word Character

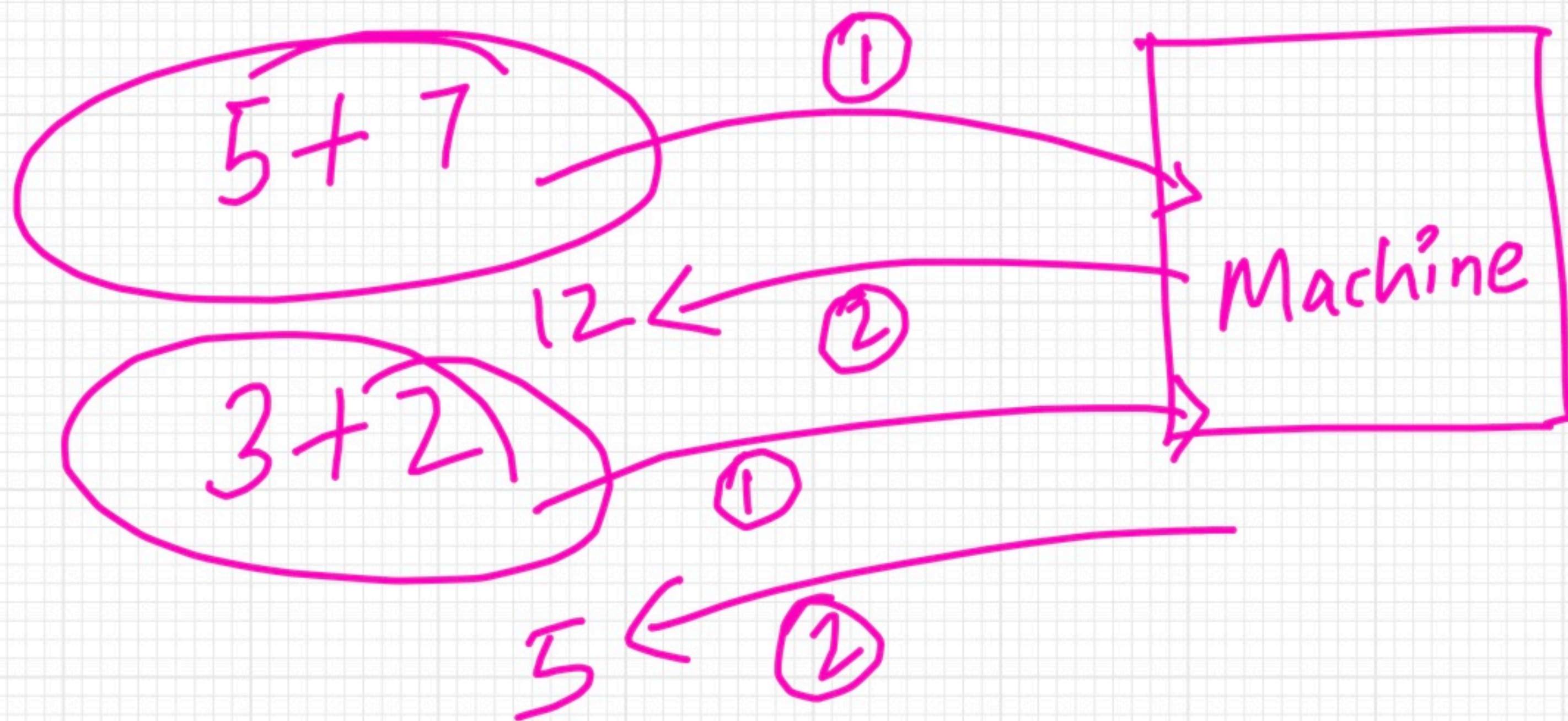
Matches any letter, digit, or underscore.
`[a-zA-Z0-9_]`

`\w{3}` matches "cat", "D_g", "123"

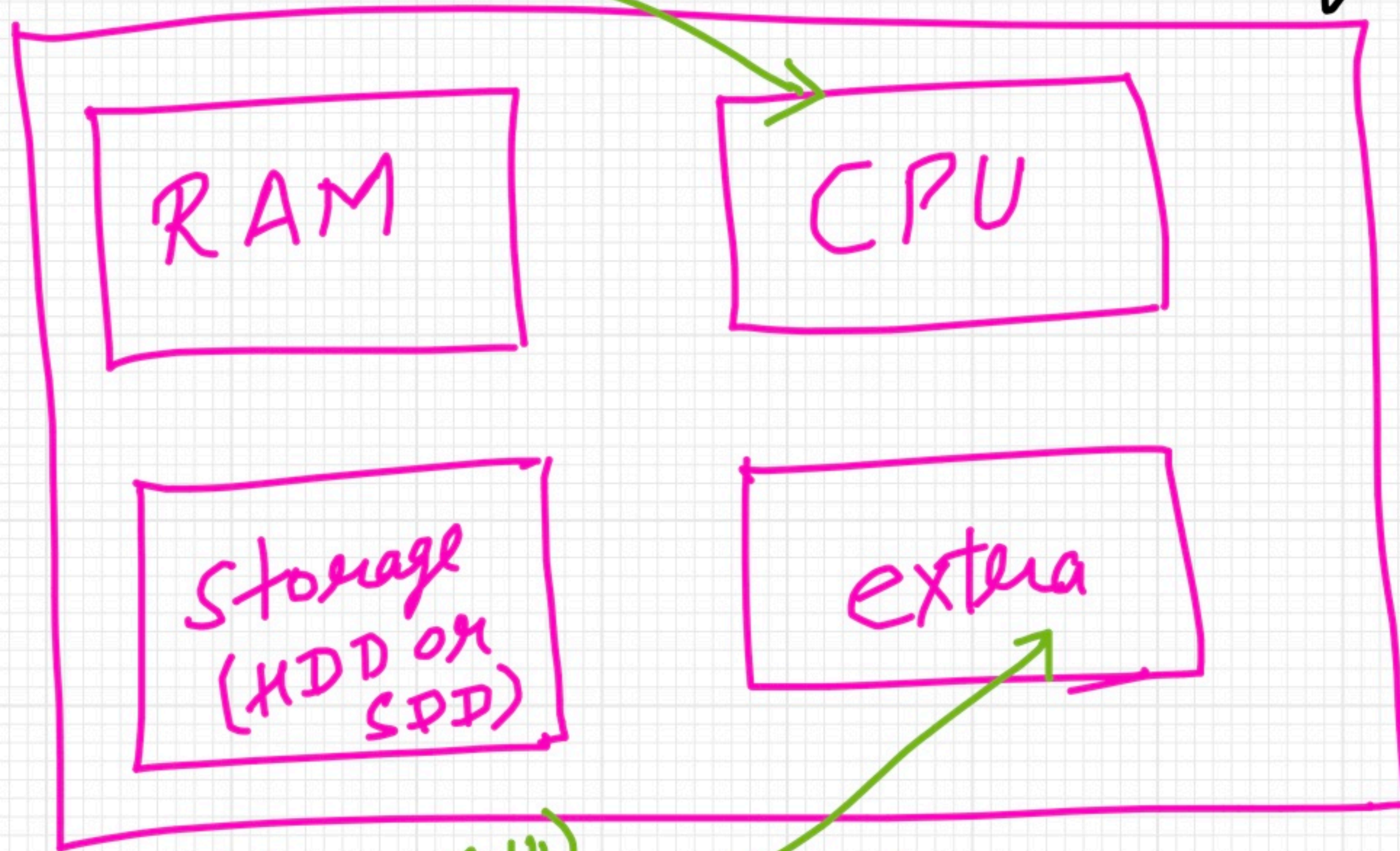
`[abc]`
`[ ]`

`[a-zA-Z0-9_]`

$$26 + 26 + 10 + 1 = 63$$



5+1 Compute task ✓ → Multiprocessing



print("hello world")
API File IO IO Task → Multithreading

Multiprocessing

